

```

procedimento exibir_menu()

    escreva("=====")
    escreva("PROJETO AV01 - ESCOLHA UMA OPCAO")
    escreva("=====")
    escreva("1. Questao 1: Contagem de Ocorrencias")
    escreva("2. Questao 2: Analise Triangular")
    escreva("3. Questao 3: Matrices 3D")
    escreva("4. Questao 4: Analise Assimetrica")
    escreva("5. Questao 5: Busca Binaria")
    escreva("0. Sair")
    escreva("=====")
    escreva("Digite sua escolha: ")

fim procedimento

```

```

procedimento exibir_header()

    escreva("=====")
    escreva("COMPLEXIDADE E COMPUTABILIDADE")
    escreva("PROFESSOR: HERRIOTR")
    escreva("PROJETO AV01")
    escreva("")
    escreva("MEMBROS DO GRUPO:")
    escreva("1. [Diego Feitosa]")
    escreva("2. [Diego Braga]")
    escreva("3. [Jose Kleyton]")
    escreva("4. [Lucas Espindola]")
    escreva("=====")

fim procedimento

```

funcao principal()

// Inicializa a semente para geração de números aleatórios

inicializar_gerador_aleatorio(tempo_atual)

inteiro opcao

chamar exibir_header()

repita

chamar exibir_menu()

leia(opcao)

escolha (opcao)

caso 1:

escreva("=== QUESTAO 1 ===")

chamar executar_q1()

caso 2:

escreva("=== QUESTAO 2 ===")

chamar executar_q2()

caso 3:

escreva("=== QUESTAO 3 ===")

chamar executar_q3()

caso 4:

escreva("=== QUESTAO 4 ===")

chamar executar_q4()

caso 5:

 escreva("=== QUESTAO 5 ===")

 chamar executar_q5()

caso 0:

 escreva("Encerrando...")

caso contrario:

 escreva("Opcao invalida!")

fim escolha

enquanto (opcao != 0)

 retorne 0

fim função

funcao ler_opcao_entrada() retorna inteiro

 inteiro opcao

 escreva("Deseja entrada manual (1) ou aleatoria (2)? ")

 leia(opcao)

 retorne opcao

fim funcao

procedimento preencher_vetor_manual(vetor, inteiro tamanho)

 para i de 0 ate tamanho - 1 faca

 escreva("Digite o elemento [", i, "]: ")

```

        leia(vetor[i])
    fim para
fim procedimento

procedimento preencher_vetor_aleatorio(vetor, inteiro tamanho)
    para i de 0 ate tamanho - 1 faca
        // Gera um número aleatório entre 1 e 100
        vetor[i] <- (aleatorio() mod 100) + 1
    fim para
fim procedimento

procedimento exibir_vetor(vetor, inteiro tamanho, texto nome)
    escreva(nome, " (tamanho ", tamanho, "):")
    para i de 0 ate tamanho - 1 faca
        escreva_mesma_linha(vetor[i], " ")
    fim para
    escreva("") // Quebra de linha
fim procedimento

procedimento contagem(inteiro n, inteiro k, vetorInteiro, listaElementos)
    inteiro somaTotal <- 0

    chamar exibir_vetor(vetorInteiro, n, "Vetor Principal")
    chamar exibir_vetor(listaElementos, k, "Vetor Buscados")

    escreva("RESULTADO:")

    para i de 0 ate k - 1 faca

```

```

inteiro contagem_elemento <- 0

para j de 0 ate n - 1 faca
    se (vetorInteiro[j] == listaElementos[i]) entao
        contagem_elemento <- contagem_elemento + 1
    fim se
fim para

escreva("[", listaElementos[i], " aparece ", contagem_elemento, " vez(es)]")

somaTotal <- somaTotal + contagem_elemento

fim para

escreva("Soma total: ", somaTotal)

fim procedimento

procedimento executar_q1()

    inteiro n, k

    escreva("Digite o tamanho do vetor principal (n): ")
    leia(n)
    escreva("Digite a quantidade de buscados (k): ")
    leia(k)

    // Declaração dos vetores com os tamanhos informados
    inteiro vetorInteiro[n]
    inteiro listaElementos[k]

    inteiro opcao <- chamar ler_opcao_entrada()

    se (opcao == 1) entao

```

```

    escreva("Preenchendo vetor principal manualmente:")
    chamar preencher_vetor_manual(vetorInteiro, n)

    escreva("Preenchendo vetor de buscados manualmente:")
    chamar preencher_vetor_manual(listaElementos, k)
senao
    chamar preencher_vetor_aleatorio(vetorInteiro, n)
    chamar preencher_vetor_aleatorio(listaElementos, k)
    escreva("Dados preenchidos aleatoriamente!")
fim se

    chamar contagem(n, k, vetorInteiro, listaElementos)
fim procedimento

```

```

procedimento preencher_matriz_manual(inteiro n, inteiro m, matriz)
    para i de 0 ate n - 1 faca
        para j de 0 ate m - 1 faca
            escreva("Digite matriz[" , i, "][", j, "]: ")
            leia(matriz[i][j])
        fim para
    fim para
fim procedimento

```

```

procedimento preencher_matriz_aleatorio(inteiro n, inteiro m, matriz)
    para i de 0 ate n - 1 faca
        para j de 0 ate m - 1 faca
            // Gera um número aleatório entre 1 e 100
            matriz[i][j] <- (aleatorio() mod 100) + 1
        fim para
    fim para

```

```

        fim para
    fim para
fim procedimento

procedimento exibir_matriz(inteiro n, inteiro m, matriz)
    escreva("Matriz (", n, "x", m, "):")
    para i de 0 ate n - 1 faca
        para j de 0 ate m - 1 faca
            escreva_mesma_linha(matriz[i][j], " ") // Mantém os elementos alinhados na
            mesma linha
        fim para
        escreva("\n") // Quebra de linha para a próxima linha da matriz
    fim para
fim procedimento

```

```

funcao multiplo5(inteiro n, matriz) retorna inteiro
    chamar exibir_matriz(n, n, matriz)

```

```

inteiro mult5 <- 0

```

```

    escreva("Verificacoes (i <= j):")
    para i de 0 ate n - 1 faca
        para j de 0 ate n - 1 faca
            se (i <= j) entao
                inteiro soma <- matriz[i][j] + matriz[j][i]

```

```

        // O operador 'mod' retorna o resto da divisão

```

```

        se (soma mod 5 == 0) entao

```

```
        escreva("matriz[" , i, "]" , j, "] + matriz[" , j, "]" , i, "] = " , matriz[i][j], " + " ,  
matriz[j][i], " = " , soma, " (multiplo de 5)")
```

```
        mult5 <- mult5 + 1
```

```
    fim se
```

```
    fim se
```

```
    fim para
```

```
    fim para
```

```
    retorne mult5
```

```
    fim funcao
```

```
procedimento executar_q2()
```

```
    inteiro n, opcao, resultado
```

```
    escreva("Digite o tamanho da matriz (n x n): ")
```

```
    leia(n)
```

```
// Declaração da matriz quadrada com dimensões n x n
```

```
    inteiro matriz[n][n]
```

```
    escreva("Deseja entrada manual (1) ou aleatoria (2)? ")
```

```
    leia(opcao)
```

```
    se (opcao == 1) entao
```

```
        escreva("Preenchendo matriz manualmente:")
```

```
        chamar preencher_matriz_manual(n, n, matriz)
```

```
    senao
```

```
        chamar preencher_matriz_aleatorio(n, n, matriz)
```



```
        escreva("Matriz preenchida aleatoriamente!")
    fim se

    resultado <- chamar multiplo5(n, matriz)

    escreva("RESULTADO:")
    escreva("Total de multiplos de 5: ", resultado)
fim procedimento
```

```
procedimento preencher_matriz3d_manual(inteiro n, matriz)
    para i de 0 ate n - 1 faca
        para j de 0 ate n - 1 faca
            para k de 0 ate n - 1 faca
                escreva("Digite matriz[", i, "][", j, "][", k, "]: ")
                leia(matriz[i][j][k])
            fim para
        fim para
    fim para
fim procedimento
```

```
procedimento preencher_matriz3d_aleatorio(inteiro n, matriz)
    para i de 0 ate n - 1 faca
        para j de 0 ate n - 1 faca
            para k de 0 ate n - 1 faca
                matriz[i][j][k] <- (aleatorio() mod 100) + 1
            fim para
        fim para
    fim para
```

fim procedimento

procedimento exibir_matriz3d_resumido(inteiro n, matriz)

 escreva("Matriz (", n, "x", n, "x", n, ") - Primeiros elementos:")

 // Define o limite de camadas a exibir (análogo à condição $n < 2 ? n : 2$)

 inteiro limite_camadas

 se ($n < 2$) entao

 limite_camadas <- n

 senao

 limite_camadas <- 2

 fim se

 para i de 0 ate limite_camadas - 1 faca

 escreva("Camada ", i, ":")

 para j de 0 ate n - 1 faca

 para k de 0 ate n - 1 faca

 escreva_mesma_linha(matriz[i][j][k], " ")

 fim para

 escreva("") // Quebra de linha para a linha da matriz

 fim para

 escreva("") // Quebra de linha para separar as camadas

 fim para

fim procedimento

funcao comparaMatrizes3D(inteiro n, matrizA, matrizB) retorna inteiro

 chamar exibir_matriz3d_resumido(n, matrizA)

```

// Usando inteiro_longo para evitar overflow em somas grandes

inteiro_longo somaA <- 0
inteiro_longo somaB <- 0

para i de 0 ate n - 1 faca
  para j de 0 ate n - 1 faca
    para k de 0 ate n - 1 faca
      somaA <- somaA + matrizA[i][j][k]
      somaB <- somaB + matrizB[i][j][k]
    fim para
  fim para
fim para

escreva("RESULTADO:")
escreva("Soma A: ", somaA)
escreva("Soma B: ", somaB)

se (somaA >= somaB) entao
  escreva("A >= B? SIM (retorna 1)")
  retorne 1
senao
  escreva("A >= B? NAO (retorna 0)")
  retorne 0
fim se

fim funcao

procedimento executar_q3()

inteiro n, opcao

```

```
escreva("Digite o tamanho da matriz (n x n x n): ")
```

```
leia(n)
```

```
// Declaração das matrizes tridimensionais
```

```
inteiro matrizA[n][n][n]
```

```
inteiro matrizB[n][n][n]
```

```
escreva("Deseja entrada manual (1) ou aleatoria (2)? ")
```

```
leia(opcao)
```

```
se (opcao == 1) entao
```

```
    escreva("Preenchendo matriz A manualmente:")
```

```
    chamar preencher_matriz3d_manual(n, matrizA)
```

```
    escreva("Preenchendo matriz B manualmente:")
```

```
    chamar preencher_matriz3d_manual(n, matrizB)
```

```
senao
```

```
    chamar preencher_matriz3d_aleatorio(n, matrizA)
```

```
    chamar preencher_matriz3d_aleatorio(n, matrizB)
```

```
    escreva("Matrizes preenchidas aleatoriamente!")
```

```
fim se
```

```
    chamar comparaMatrizes3D(n, matrizA, matrizB)
```

```
fim procedimento
```

```
funcao calcular_fatorial(inteiro n) retorna inteiro_longo
```

```
    se (n <= 1) entao
```

retorne 1

fim se

inteiro_longo fat <- 1

para i de 2 ate n faca

 fat <- fat * i

fim para

retorne fat

fim funcao

funcao processar_vetor(inteiro n, vetor) retorna inteiro_longo

 escreva("Vetor (tamanho ", n, "):")

 // Define limite de exibição para não poluir a tela se o vetor for muito grande

 inteiro limite_exibicao

 se (n < 50) entao

 limite_exibicao <- n

 senao

 limite_exibicao <- 50

 fim se

 para i de 0 ate limite_exibicao - 1 faca

 escreva_mesma_linha(vetor[i], " ")

 fim para

 se (n > 50) entao

 escreva_mesma_linha("...")

fim se

escreva("") // Quebra de linha

escreva("Processamento:")

inteiro_longo somatorio <- 0

para i de 0 ate n - 1 faca

 // Verifica se o número é par

 se (vetor[i] mod 2 == 0) entao

 escreva("vetor[", i, "] = ", vetor[i], " (PAR) -> soma: +", vetor[i])

 somatorio <- somatorio + vetor[i]

 senao

 inteiro_longo fat <- chamar calcular_fatorial(vetor[i])

 escreva("vetor[", i, "] = ", vetor[i], " (IMPAR) -> fatorial: ", fat)

 somatorio <- somatorio + fat

 fim se

fim para

retorne somatorio

fim funcao

procedimento executar_q4()

 inteiro n, opcao

 escreva("Digite o tamanho do vetor (n): ")

 leia(n)

```
inteiro vetor[n]
```

```
escreva("Deseja entrada manual (1) ou aleatoria (2)? ")
```

```
leia(opcao)
```

```
se (opcao == 1) entao
```

```
    escreva("Preenchendo vetor manualmente:")
```

```
    para i de 0 ate n - 1 faca
```

```
        escreva("Digite o elemento [", i, "]: ")
```

```
        leia(vetor[i])
```

```
    fim para
```

```
senao
```

```
    para i de 0 ate n - 1 faca
```

```
        // Valores entre 1 e 15 para evitar que o fatorial passe do limite numérico
```

```
        vetor[i] <- (aleatorio() mod 15) + 1
```

```
    fim para
```

```
    escreva("Vetor preenchido aleatoriamente!")
```

```
fim se
```

```
inteiro_longo resultado <- chamar processar_vetor(n, vetor)
```

```
escreva("RESULTADO:")
```

```
escreva("Somatorio: ", resultado)
```

```
fim procedimento
```

```
funcao busca_binaria(inteiro n, vetor, inteiro alvo) retorna inteiro
```

```
    inteiro inicio <- 0
```

```
inteiro fim <- n - 1
```

```
enquanto (inicio <= fim) faca
```

```
    inteiro meio <- inicio + (fim - inicio) / 2
```

```
    se (vetor[meio] == alvo) entao
```

```
        retorne 1
```

```
    fim se
```

```
    se (vetor[meio] < alvo) entao
```

```
        inicio <- meio + 1
```

```
    senao
```

```
        fim <- meio - 1
```

```
    fim se
```

```
fim enquanto
```

```
retorne 0
```

```
fim funcao
```

```
procedimento ordenar_vetor(vetor, inteiro n)
```

```
    // Implementação do algoritmo Bubble Sort
```

```
    para i de 0 ate n - 2 faca
```

```
        para j de 0 ate n - i - 2 faca
```

```
            se (vetor[j] > vetor[j+1]) entao
```

```
                inteiro temp <- vetor[j]
```

```
                vetor[j] <- vetor[j+1]
```

```
                vetor[j+1] <- temp
```

```
            fim se
```



```

        fim para
    fim para
fim procedimento

funcao contar_elementos(inteiro n, vetorA, vetorB) retorna inteiro
    escreva("Vetor A (nao ordenado) - primeiros 20 elementos:")

    inteiro limite_exibicao
    se (n < 20) entao
        limite_exibicao <- n
    senao
        limite_exibicao <- 20
    fim se

    para i de 0 ate limite_exibicao - 1 faca
        escreva_mesma_linha(vetorA[i], " ")
    fim para

    se (n > 20) entao
        escreva_mesma_linha("...")
    fim se

    escreva("") // Quebra de linha

    escreva("Vetor B (ordenado) - primeiros 20 elementos:")
    para i de 0 ate limite_exibicao - 1 faca
        escreva_mesma_linha(vetorB[i], " ")
    fim para

```

se (n > 20) entao

 escreva_mesma_linha("...")

fim se

escreva("") // Quebra de linha

escreva("Processando...")

inteiro total_encontrados <- 0

para i de 0 ate n - 1 faca

 // Se a busca binária retornar 1 (verdadeiro), incrementa o total

 se (chamar busca_binaria(n, vetorB, vetorA[i]) == 1) entao

 total_encontrados <- total_encontrados + 1

 fim se

fim para

retorne total_encontrados

fim funcao

procedimento executar_q5()

 inteiro n, opcao

 escreva("Digite o tamanho dos vetores (n): ")

 leia(n)

 inteiro vetorA[n]

 inteiro vetorB[n]

```
escreva("Deseja entrada manual (1) ou aleatoria (2)? ")
```

```
leia(opcao)
```

```
se (opcao == 1) entao
```

```
    escreva("Preenchendo vetor A manualmente:")
```

```
    para i de 0 ate n - 1 faca
```

```
        escreva("Digite o elemento [", i, "]: ")
```

```
        leia(vetorA[i])
```

```
    fim para
```

```
    escreva("Preenchendo vetor B manualmente (DEVE ESTAR ORDENADO):")
```

```
    para i de 0 ate n - 1 faca
```

```
        escreva("Digite o elemento [", i, "]: ")
```

```
        leia(vetorB[i])
```

```
    fim para
```

```
senao
```

```
    para i de 0 ate n - 1 faca
```

```
        vetorA[i] <- (aleatorio() mod 200) + 1
```

```
        vetorB[i] <- (aleatorio() mod 200) + 1
```

```
    fim para
```

```
    // Garante que o vetor B esteja ordenado para a busca binária funcionar
```

```
    chamar ordenar_vetor(vetorB, n)
```

```
    escreva("Vetores preenchidos aleatoriamente! Vetor B foi ordenado.")
```

```
fim se
```

```
inteiro resultado <- chamar contar_elementos(n, vetorA, vetorB)
```

```
escreva("RESULTADO:")
```

```
escreva("Total de elementos de A encontrados em B: ", resultado)
```

```
fim procedimento
```